

Document number: P1846R0

Date: 2019-08-05

Reply-to: John McFarlane, wg21@john.mcfarlane.name

Audience: SG20

Teach `class` Last

Abstract

Classes are at the heart of type safety in C++ and their utility will long outlive the OOP era. But there's a huge difference between designing a new class and utilising an existing class. As experts, we take for granted just how much knowledge is required to safely write a class. And with every revision of the standard, the need for users to write their own classes decreases. I propose that teaching of class design should be treated as an advanced skill which comes after a long period of using and appreciating the classes provided by the standard library.

Introduction

[P1389R0](#) entitled "Standing Document for SG20: Guidelines for Teaching C++ to Beginners", lists beginner topics in four stages. The first stage, "fundamentals", lists 24 topics including "designing and using classes". This suggests that designing classes should be taught early in a student's curriculum.

While I agree that C++ is next to useless without the ability to use classes, I argue that class design is increasingly difficult and unnecessary and should be left until far later.

I identify three influences which, above all others, have led me to this conclusion.

Firstly, advocates of modern and functional programming often discourage the use of class hierarchies as the route to API design. For example, "[Inheritance Is The Base Class of Evil](#)", leaves little room for interpretation.

Secondly, when preparing introductory material for presentation to engineers who are new to C++, it is easy to imagine a 60-minute talk on classes which could make them 'dangerous'. It is much harder to imagine a 180-minute talk that would make them 'safe'. Topics such as good API design, resource management, object lifetime and special functions all require careful explanation. The number of keywords alone which are devoted to writing a class is dizzying.

Thirdly, on several occasions, I have been asked an architectural question regarding a design which is heavily object oriented. In each case, all the classes could be replaced with functional programming paradigms and this reduced the cognitive load necessary to resolve the question.

If knowledge of class design is to be deferred, this leaves a gap which must be filled. Most of the remainder of the paper explains how this might be done.

The Separate Concerns of a Class

I find it ironic that a programming construct so closely related to the S.O.L.I.D. principles should itself embody so many concerns. This section explores some of the functionality a class can provide and why it is not necessary to duplicate that functionality in a new class.

Polymorphism

Dynamic polymorphism is the headline feature which classes provide. (It is a given that static polymorphism is often a better choice.) Assuming a clean design involving a clear division between interface and implementation, a great deal of boilerplate code is necessary. As an understanding of interface purity develops, a base class will refine down to two virtual functions: a 'doing' function and a destructor.

At that point, what we find ourselves with is a verbose reimplementations of `function` without the value semantics. On rare occasions, a statically allocated object of an implementation class can outperform a `function` object. Conversely, SBO means that the opposite can also be true.

Composition

Classes can be used to group objects together. But increasingly, there are simpler ways to achieve this. A `struct` is the simplest example. Where the components require no names, `tuple` is ideal.

RAII

RAII continues to distinguish C++ from almost every other language. But its power is now largely hidden from plain sight behind value types. The vast majority of instances where an object manages a resource, that resource is free store memory and for this, we have `unique_ptr`. Most of what is left is covered by `unique_resource`.

Organization

A common abuse of classes is as containers for definitions. This is probably an interloper from OO languages such as Java. Advice such as rule 44 of [C++ Coding Standards](#): "Prefer writing nonmember nonfriend functions" is a great coverall which discourages this bad practices — among others. Regardless, we have namespaces for grouping definitions together.

In-Place Polymorphism

In situations where one might think to use `dynamic_cast` the first reaction is: you probably don't want to do that. But there are situations where alternatives are not representable via an interface. In such cases, `any` or `variant` might be what you're looking for.

Type-Safe Aliases

There are situations where a user would like a carbon copy of an existing class but with distinct semantics. For example:

```
using sanitized_string = string;
sanitized_string sanitize(string unsanitized);

sanitized_string good{sanitize("<script>alert('pwned!');</script>")};
sanitized_string bad{"<script>alert('pwned!');</script>"};
```

Here the compiler doesn't stop the user from instantiating insecure object `bad`. If `sanitized_string` instead inherits from `string`

```
class sanitized_string : string {};
```

a degree of type safety is introduced. Whether or not this is a sensible use of inheritance, it is one reason why users might consider defining a class.

Function Objects

The traditional way to create a function object is to define a class with an overloaded function call operator. Lambda expressions largely replace this need with a more terse syntax.

Encapsulation

Restricting access to variables can be achieved in various ways. The `private` keyword is just one of these. Function-level encapsulation is a very powerful alternative which is often overlooked because of its ubiquity. Lambda capture follows naturally from function-level encapsulation and often do precisely the same job as class-based encapsulation.

Invariants

Enforcing an invariant through use of access specifiers remains a compelling use case for a class. I'm talking here about the case where one or more variables must be in a subset of their possible states. Where the state isn't expected to change, the same effect can be achieved through the `const` type qualifier. Understanding `const` should be prioritized over `class` and `private`.

So When Then?

One thing to be observed about the above laundry list of features is that many of them are implemented using classes. There is no contention that classes remain a powerful and necessary abstraction. One might even characterise functional programming as OOP's logical conclusion.

At some point, the student will find a gap in the set of language and library features. It is the job of WG21 to plug these gaps in descending order of occurrence. Until then, they have reason to define a class of their own.

Having learned all of the above programming techniques — some of which depend on library classes — the student should be in a good position to appreciate when is a good time to take the plunge and author a class.

I have concluded that invariants are the remaining 'killer app' for the user-land class definition. Invariants make an excellent way to frame the topic of good API design — one of the hardest skills a C++ programmer will ever learn. This is the time when I believe that the topic should be broached.

Conclusion

Given C++'s heritage, it would be optimistic to expect the `class` keyword to be pushed straight to stage 3 of the standing document. It would also be premature and extreme: WG21 is not finished with the task of separating the concerns of classes into features of the IS.

But I believe we should start to question whether class design is still central to the user's ability to write C++ programs. Section 2.3 of the standing document is entitled [\[delay\] Delay features until there is a genuine use-case](#). I think it is clear that the list of genuine use-cases for writing a class is dwindling.